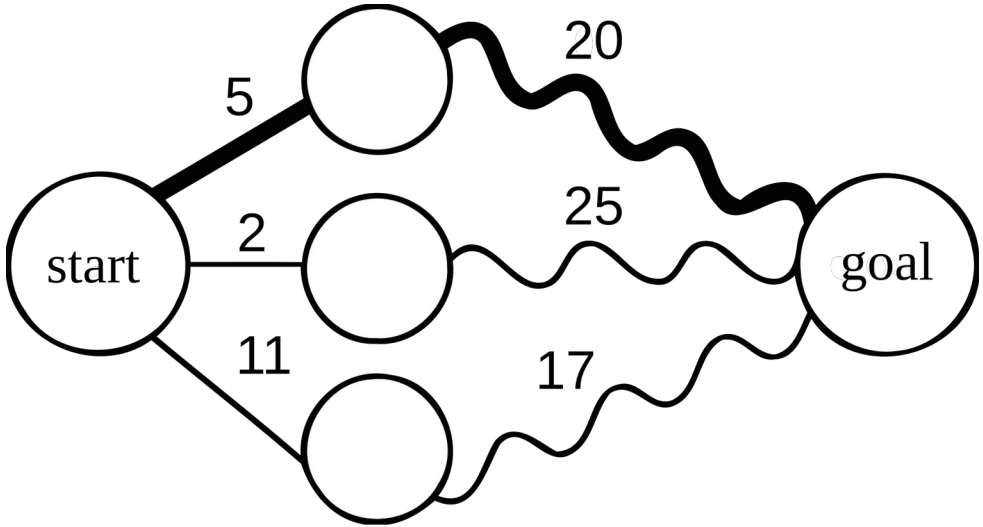
Lecture 17

Dynamic Programming III



Announcements

- Office hours are up on course website
  - T: 12pm-4pm; F: 11am-1pm
  - Schedule an appointment individually or as a group

- Project II teams and topics released at the end of class
  - Deliverable 1 due 11/13

Last time

Extracting an optimal solution from the iterative algorithm

Metal Cutting Problem

- Recursive algorithm

Today

Metal Cutting Problem

- Recurrence relation
- Bottom-up approach

Knapsack Problem

- Pseudo-polynomial complexity

Metal Cutting Problem

Version of a classic problem in operations research

Input:

- Steel rod of length `n`, which can be cut into integer lengths and sold for price p_i for length `i`

Goal:

- Subdivide steel rod to maximize sell price

Dynamic Programming Recipe

- Step 1: Devise simple recursive algorithm for the value of the optimal solution
- Step 2: Write recurrence relation for optimal value
- Step 3: Design bottom-up iterative algorithm
- Step 4: Recover optimal solution

Recursive Algorithm

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

base case

consider all possible next cuts to make

Recurrence Relation

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

$$OPT(m) = \begin{cases} 0 & \text{if } m = 0 \\ max_{1 \le i \le m} \{ p_i + OPT(m-i) \} & \text{if } m > 0 \end{cases}$$

Recurrence Relation

```
CutMetal(n):
    if n = 0:
        return 0
    poss = []
    for i = 1 to n:
        poss.append(p[i] + CutMetal(n-i))
    return max(poss)
```

$$OPT(m) = \begin{cases} 0 & \text{if } m = 0 \\ max_{1 \le i \le m} \{ p_i + OPT(m-i) \} & \text{if } m > 0 \end{cases}$$

- The recurrence provides all info needed to design an iterative algorithm
- Start from 0 then iterate upward

Iterative Algorithm

```
CutMetalItr(n):
    L = []
    L[0] = 0
    for i = 1 to n:
        v = 0
        for j = 1 to i:
            v = max(v, p[j] + L[i-j])
        L[i] = v
    return L
```

Iterative Algorithm

```
CutMetalItr(n):
    L = []
    L[0] = 0
    for i = 1 to n:
        v = 0
        for j = 1 to i:
            v = max(v, p[j] + L[i-j])
        L[i] = v
    return L
```

Running time: $\Theta(n^2)$

Retrieve Optimal Solution

```
CutMetalItr(n):
    L = []
    B = []
    L[0] = 0
    B[0] = 0
    for i = 1 to n:
        v = 0
        idx = 0
        for j = 1 to i:
            if p[j] + L[i-j] > v:
                v = p[j] + L[i-j]
                idx = j
        L[i] = v
        B[i] = idx
    return L, B
```

Retrieve Optimal Solution

```
CutMetalItr(n):
    L = []
    B = []
    L[0] = 0
    B[0] = 0
    for i = 1 to n:
        v = 0
        idx = 0
        for j = 1 to i:
            if p[j] + L[i-j] > v:
                v = p[j] + L[i-j]
                idx = j
        L[i] = v
        B[i] = idx
    return L, B
```

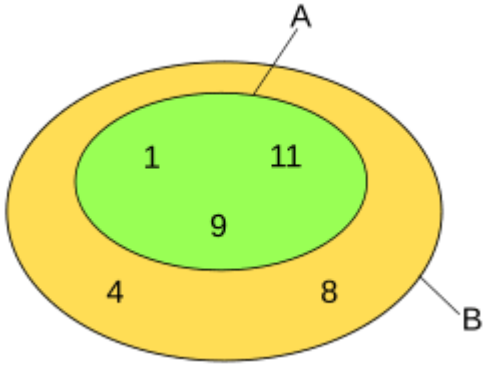
```
getSolution(M):
    given lists L, B
    A = {}
    i = n
    while i > 0:
        j = B[i]
        A.add(j)
        i = i - j
    return A
```

Subset Sum Problem

Input:

- Items $1, \ldots, n$
- Weights for w_i each item (integers)
- Capacity W

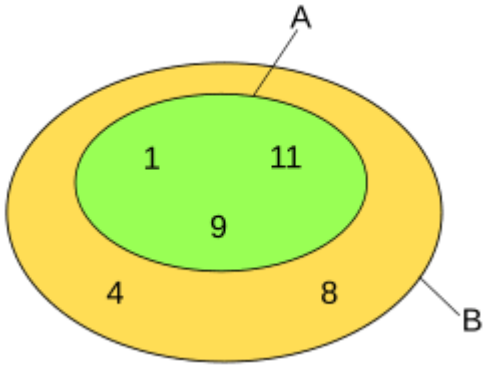
Goal: Select subset of items S whose total weight is as large as possible without exceeding the capacity



Subset Sum Problem

Input:

- Items $1, \ldots, n$
- Weights for w_i each item (integers)
- Capacity W



Goal: Select subset of items S whose total weight is as large as possible without exceeding the capacity

Running time of bottom-up algorithm:

- $O(nW)$

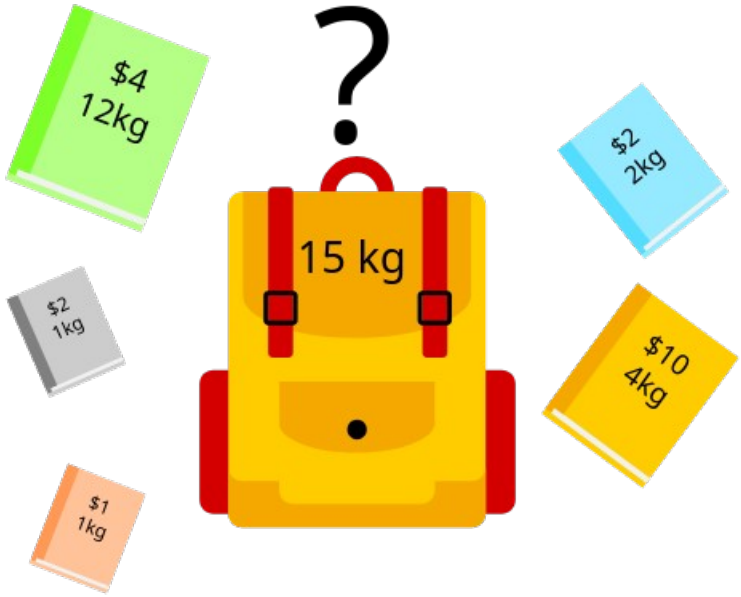
Called pseudo-linear since it is linear for any fixed W

Knapsack Problem

Input:

- Items $1, \ldots, n$
- Weights for w_i each item (integers)
- Values v_i for each item (integers)
- Capacity W

Goal: Select subset of items S whose total value is as large as possible without total weight exceeding the capacity



Next Time

- Search algorithms